

이번 과제는 중복 연산자를 이용하여서 기존에 만들었던 stl에 적용을 시켜보는 과제이다.

본 과제를 하기 이전에 먼저 operator에 대해서 이해하기 위해 교육자료에 있는 코드를 작성해 보았다.

```
enum ColorType { RED, BLUE, GREEN, VIOLET, ORANGE, UNKNOWN };

class Color {
    ColorType type;
public:
    Color(ColorType t = UNKNOWN) : type(t) {}

    // 핵심: + 연산자를 오버로딩해서 두 색을 섞음
    Color operator+(const Color& other) const {
        if ((type == BLUE && other.type == RED) ||
            (type == RED && other.type == BLUE))
            return Color(VIOLET);
        if ((type == RED && other.type == GREEN) ||
            (type == GREEN && other.type == RED))
            return Color(ORANGE);
        return Color(UNKNOWN);
    }

    std::string name() const {
        switch (type) {
            case RED:    return "RED";
            case BLUE:   return "BLUE";
            case GREEN:  return "GREEN";
            case VIOLET: return "VIOLET";
            case ORANGE: return "ORANGE";
            default:     return "UNKNOWN";
        }
    }
};

int main(void) {
    Color a(BLUE), b(RED), c;
    c = a + b; // operator+ 호출
    std::cout << c.name() << "\n"; // VIOLET
    return 0;
}
```

처음 교육자료만 보았을 때는 operator를 선언만하면 C++자체의 기본 라이브러리에서 RED BLUE 색을 섞어서 VIOET이 나오는 것을 컴퓨터적 연산을 통해서 나오는 줄 알고 신기해서 실습코드를 작성해 보았지만 사실은 사용자가 직접 코드로 각각의 매개변수들을 판단해서 출력을 하는 것이었다.

아무튼 이 실습코드를 통해 operator의 중복연산자에 대해 어느정도 이해하고 과제를 수행할 수 있었다.

```
cdh operator+(const cdh& other) const {
    cdh result;
    size_t n = (this->size_ > other.size_) ? this->size_ : other.size_;
    for (size_t i = 0; i < n; i++) {
        T sum = T(); // int 면 0
        if (i < this->size_) sum += this->data_[i];
        if (i < other.size_) sum += other.data_[i];
        result.push_back(sum);
    }
    return result; // 값 반환 → 복사 생성자(또는 최적화)
}
```

이 코드는 구현 벡터의 operator+ 코드이다. 이 코드를 이용하여서 v0 벡터와 v1벡터를 더해서 새로운 벡터에 할당할 수 있게 되었다.

<pre>[v0] > push_back 10 push_back(10) ---- 전체 객체 (1개, * = 현재) ---- * v0 [10], size=1 ----- [v0] > push_back 20 push_back(20) ---- 전체 객체 (2개, * = 현재) ---- * v0 [10 20], size=2 ----- [v0] > new new() -> v1 생성 및 전환 ---- 전체 객체 (2개, * = 현재) ---- v0 [10 20], size=2 * v1 [], size=0 ----- [v1] > push_back 1 push_back(1) ---- 전체 객체 (2개, * = 현재) ---- v0 [10 20], size=2 * v1 [1], size=1 -----</pre>	<pre>[v1] > push_back 2 push_back(2) ---- 전체 객체 (2개, * = 현재) ---- v0 [10 20], size=2 * v1 [1 2], size=2 ----- [v1] > push_back 3 push_back(3) ---- 전체 객체 (2개, * = 현재) ---- v0 [10 20], size=2 * v1 [1 2 3], size=3 ----- [v1] > add 0 1 add: v0 + v1 -> v2 생성 ---- 전체 객체 (3개, * = 현재) ---- v0 [10 20], size=2 v1 [1 2 3], size=3 * v2 [11 22 3], size=3 -----</pre>
--	---

<실행결과>

Day2와 다르게 이제는 벡터를 여러 개 생성할 수 있게 수정하였다. 그래서 단순히 add

함수만 구현하였지만, 함수적으로 중복연산자를 추가로 구현할 수 있게 해놓았다.

두 객체의 같은 index값 데이터끼리 +연산자를 사용하였고 두 객체의 size_값이 다르면 없는 데이터값은 0으로 바뀌어서 연산자를 실행시켰다.